POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Executive Summary of the Thesis

Space Cloud: Autonomous Live Container Migration for Stateful Edge AI in LEO Satellite Constellations

Laurea Magistrale in Computer Science and Engineering - Ingegneria Informatica

Author: Alessandro Guazzi

Advisor: Prof. Emanuele Della Valle

Academic year: 2025-2026

Abstract. When a wildfire breaks out, real-time satellite tracking provides critical operational intelligence. Running AI fire-prediction models directly in space avoids sending massive raw files to the ground, but pushes satellite processors to their physical limits. Without air to dissipate heat, processors operating in vacuum overheat within minutes of sustained computation. When standard Kubernetes health-check timers expire during thermal throttling, the orchestrator flags the healthy pod as unresponsive and terminates it, wiping out days of in-memory observation history and disrupting the prediction model. Space Cloud migrates these live workloads moments before hardware failure. By tuning host kernel parameters and configuring container runtime interfaces, the platform migrates active fire-tracking memory to a cooler neighboring satellite over inter-satellite links. Splitting the workload into a stateful history sidecar and a stateless inference engine cuts the transfer payload by $\approx 85\%$. A model predictive controller tracks local temperature and battery reserves, executing migration before processors trigger the 95 °C hardware shutdown. To route the payload safely, a logarithmic scoring function evaluates thermal and energy margins, assigning satellites near operational limits prohibitively high penalties. Across an extensive experimental campaign combining factorial stress trials and a continuous 23-minute run spanning multiple simulated orbital cycles, Space Cloud maintains stateful continuity and preserves the wildfire observation history under orbital stress.

Keywords: Low Earth Orbit (LEO); Edge Computing; Live Container Migration; CRIU; Kubernetes; Wildfire Spread Prediction.

1. Introduction and Problem Definition

When a wildfire spreads, ground responders rely on constant satellite tracking. Earth observation satellites traditionally act as simple relays, capturing raw imagery and transmitting it during the brief 5-to-10-minute ground station contact window. Modern constellations resolve this latency by executing deep learning models directly on board [4]. Downlinking a 16 KB classification mask instead of a 0.9 MB raw image reduces bandwidth usage by over 50:1. While flight tests like Φ-Sat-1 have shown that neural networks can execute in space [2], orbital physics severely restrict sustained computation.

Vacuum eliminates convective cooling. Lacking surrounding air, processors rely on conduction

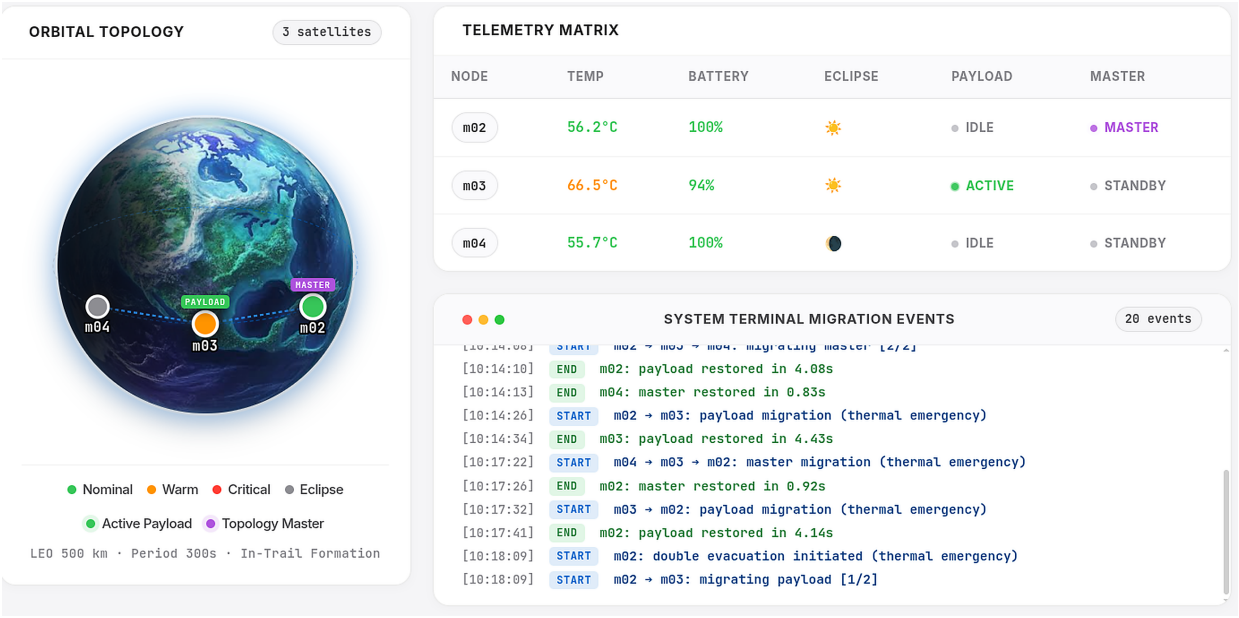


Figure 1: The Space Cloud Solution: Real-time mission control autonomously detects a thermal crisis and routes the active container state to a cooler satellite before hardware limits trigger an emergency eviction.

through heat pipes to move thermal energy to external radiators, which radiate it into space. As computational load spikes during the sunlit portion of an orbit, processors quickly hit thermal limits. At 85 °C, CPUs throttle clock speeds to prevent physical damage [2]. Furthermore, LEO satellites rely entirely on battery power during eclipses. Draining the battery triggers a hard power down, instantly erasing all volatile memory.

The wildfire prediction workload, based on the WildfireSpreadTS benchmark [3], highlights this vulnerability. The U-Net model requires a sliding historical window of five daily images. Storing past frames for multiple active fires builds a massive in-memory buffer. As processors throttle under heat, standard Kubernetes controllers miss critical check-in deadlines. The orchestrator deletes the supposedly dead container, permanently destroying the historical context. The model then waits days for new imagery to rebuild its buffer, leaving the wildfire completely unmonitored.

Space Cloud prevents this systemic failure by migrating the active state directly over the inter-satellite mesh. It leaves the heavy neural network weights behind, targeting only the vital RAM footprint holding the fire history buffers. The platform extracts this active memory, streams it to a cooler satellite, and reconstructs the running container before the source hardware hits the 95 °C thermal fuse. Completing this transfer inside the critical window between the 80 °C warning and the 95 °C cutoff preserves the workload and maintains observation continuity.

2. Related Work

Running deep learning models directly on satellites cuts downlink traffic by up to 90% [2, 4]. Most proposals assume that inter-satellite links are always fast and reliable, overlooking orbital physical realities. Under these conditions, a satellite must offload its work or shut down to protect its hardware. Managing software across many satellites requires an orchestrator like Kubernetes. Edge-focused platforms like KubeEdge split the control plane, keeping local copies of pod definitions [1]. Yet standard Kubernetes controllers still struggle with disconnections. When links drop, health-check timers expire. The orchestrator treats the silent satellite as dead and evicts its pods, terminating active workloads on operational hardware.

Keeping these services alive requires moving workloads to safe satellites before a failure occurs. Prior work explores stateless task offloading and virtual machine migration [5]. Virtual

machines are too heavy for low-power satellites, and most offloading schemes assume stateless tasks. Moving live, stateful containers between satellites remains unaddressed. State is essential for time-series applications tracking historical observations. Process-level checkpointing with CRIU can save this state by writing memory pages to disk. The main challenge is checkpoint size: CRIU captures all mapped memory, including temporary runtime data. This uncollected memory inflates the archive, making transfers over slow satellite links take longer than the remaining thermal safety margin.

3. Space Cloud: System Architecture

Space Cloud runs on top of Kubernetes to provide container migration at the LEO edge. It moves running applications between satellites without ground intervention, as illustrated in the operations dashboards in Figure 1.

3.1. Base Infrastructure and ISL Mesh

The testbed uses a four-node Kubernetes cluster. Every node runs the CRI-O runtime with checkpointing enabled. The three emulated satellite nodes are assigned separate simulated orbital planes separated by 45°. When one node's simulated temperature rises, another operates in a cooler phase of its simulated orbit. Eclipses start 37.5 seconds apart between neighboring planes, ensuring that an overheating node always has a cooler neighbor ready to take its work. The inter-satellite links connect the nodes in a line capped at 50 Mbps with a 40 ms delay per hop [5]. Configuring nodes required enabling CRI-O checkpointing, configuring CRIU to serialize active TCP sockets, aligning cgroup drivers, and disabling process namespace sharing.

3.2. Dual-Container Pod Architecture

Each workload runs as a two-container pod using the sidecar pattern. The Guardian sidecar stores compressed fire images, prediction masks, and mission metadata. It tracks each fire mission as a separate object. To save memory, the sidecar compresses history frames in RAM using DEFLATE, drastically reducing memory footprint. The Phoenix worker runs the ONNX model but keeps no permanent state. After migration, it fetches past frames from the Guardian over a local HTTP call. Because process namespace sharing is disabled, CRIU saves only the Guardian, cutting checkpoint size dimension enough to enable seamless migration.

3.3. Distributed Orchestration: The Node Agent

Each satellite runs a Node Agent as a DaemonSet to monitor health and dictate migrations. The agent runs once per second using two rules. First, if temperature reaches T_{fuse} or battery falls to B_{fuse}, the agent triggers an immediate emergency jump without waiting for predictions. Second, a local model projects temperature and battery levels 30 seconds ahead. If projections cross safe limits (T_{safe}, B_{safe}), the agent starts moving the container early. Three safeguards prevent workloads from bouncing back and forth: a 15-second cooldown timer, a mutual exclusion lock, and a rule to stay in place if no neighbor offers better conditions.

3.4. The Floating Master

The Floating Master hosts the topology database and status dashboard (Figure 1), migrating via CRIU. Topology Redis runs entirely in RAM and employs strictly synchronous I/O to ensure that CRIU can reliably serialize the database state. While migrating, the Master pauses briefly, deferring incoming triggers until restoration completes.

4. Key Implementation Mechanics

Migrating live stateful containers across low-Earth orbit requires coordinating kernel parameters, container runtimes, and graph-based routing. Standard Kubernetes treats pods as stateless, lacking native live-migration primitives. Space Cloud resolves this by configuring the host runtime: aligning Linux cgroup drivers under `systemd` for isolation, expanding kernel `inotify` watcher limits to handle rapid IPC file handshakes without event loss, and injecting custom CRIU socket configurations to prevent connection resets upon restoration. This ensures the target node matches the runtime environment.

4.1. Six-Stage Migration Pipeline

When a migration triggers, the platform executes an orchestrated six-stage Checkpoint–Transfer–Restore pipeline to ensure state preservation. (1) A file handshake over a shared `/tmp` volume directs the Guardian sidecar to flush ephemeral data, force memory garbage collection, and close sockets, explicitly shrinking the process footprint. (2) The Node Agent invokes the Kubelet Checkpoint API to dump the sidecar memory via CRIU. (3) The local deployment scales to zero to free host resources. (4) The archive streams across the ISL mesh using hop-by-hop SHA-256 verification. (5) The destination satellite natively rebuilds an OCI image wrapper via Buildah. (6) The Guardian process resumes precisely from its dumped RAM state, while the Phoenix worker cold-boots and reloads historical context via local IPC bindings.

4.2. Distributed State and Concurrency

Constellation coordination operates under severe bandwidth constraints. To prevent ISL saturation, the Node Agent delta-encodes metrics, transmitting only when temperature shifts $\geq 1.0\,^{\circ}\mathrm{C}$, battery varies $\geq 5.0\%$, or a heartbeat expires. Updates commit atomically to a shared Redis topology layer. Running Dijkstra pathfinding as an atomic Redis Lua script prevents read-write interleaving, ensuring coherent routing state. A logarithmic penalty converging to infinity near thermal (95 °C) or battery limits masks risky nodes. When a worker and Floating Master share an overheating host, sequenced evacuation moves the worker first, then the Master along an orthogonal path to prevent hotspot re-concentration.

5. Experimental Setup

The platform was evaluated using a $6 \times 3 \times 3$ factorial design with 540 trials. Six configurations isolate system features: full baseline (C1), Predictive Twin disabled (C2), Global Cooldown Guard disabled (C3), State-Sidecar Split disabled (C4), Logarithmic Utility disabled (C5), and Sequenced Evacuation disabled (C6). The evaluation tested three stress scenarios across three severity levels (Nominal, Borderline, and Correct Failure).

Scenario 1 (Link Degradation) throttles network bandwidth from 25.0 Mbps down to 4.0 Mbps. Scenario 2 (Sequential Thermal Spikes) triggers a primary migration followed by a second thermal spike on the new node, testing the 15-second cooldown timer. Scenario 3 (Double Trouble) creates a double failure: the host satellite overheats while nearby neighbors run out of battery. Hitting 95 °C starts a 5.0-second emergency timer. If the pod is not running before time expires, the run fails.

6. Experimental Results

A 540-trial campaign shows Space Cloud maintains continuity and inference performance under orbital constraints. A 3-way ANOVA indicates all tested factors (Predictive Twin, Global Cooldown, State-Sidecar Split, Logarithmic Utility, and Sequenced Evacuation) are statistically significant ($p < 0.05$). The severity of environmental stress (Nominal, Borderline, Failure) exerts the largest effect on workload survival. The impact of isolating each component is analyzed below.

6.1. Phase A: Predictive Digital Twin

Under borderline and failure conditions, the full system (C1) achieved 51.67% survival, while the reactive baseline (C2) collapsed to 5.00% ($p = 4.47 \times 10^{-9}$). Purely reactive triggers fail because CRI-O serialization and transfer overheads exceed the final thermal countdown. In double-stress trials where temperatures reached 85 °C and 92 °C, C2 recorded zero hops: the workload never initiated evacuation and was abruptly terminated by the hardware fuse. In contrast, C1 anticipated the trajectory crossing T_{safe} and migrated proactively, demonstrating that a 30-second predictive window is an essential architectural requirement for surviving orbital heating.

6.2. Phase B: Global Cooldown Guard

In the repeated thermal crisis test, disabling the cooldown lock (C3) triggered a ping-pong effect where containers migrated repeatedly between nodes, averaging 2.83 hops per run compared to 2.07 for C1 ($p = 7.55 \times 10^{-4}$). Network traffic rose from 29.97 MB to 39.52 MB ($p = 1.54 \times 10^{-3}$), threatening link saturation. As shown in Figure 2, C3 exhibits persistent

hopping across all severity regimes. Crucially, every extra hop forces a full CRIU freeze-restore cycle, creating mission blackout windows during which the inference engine is suspended and cannot process incoming wildfire imagery.

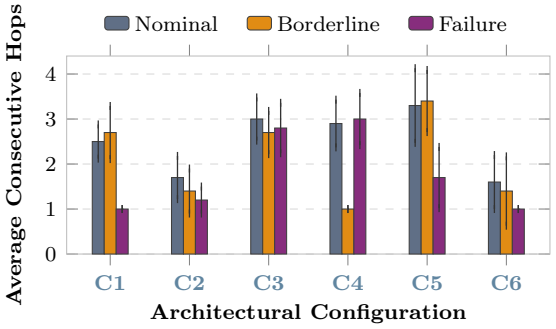


Figure 2: Effect of disabling the cooldown guard: without the 15-second cooldown lock (C_3), the orchestrator repeatedly migrates the container between satellites, causing excessive hop counts that suspend inference and interrupt the observation pipeline.

6.3. Phase C: State-Sidecar Separation

Checkpointing monolithic containers (C4) required transferring 38 MB of runtime memory versus 17 MB for the sidecar (C1). C1 finished in 25.26 s, whereas C4 averaged 31.63 s, exceeding safety margins. Consequently, C1 achieved 67.78% survival while C4 fell to 44.44%, systematically piercing the safety horizon (Figure 3).

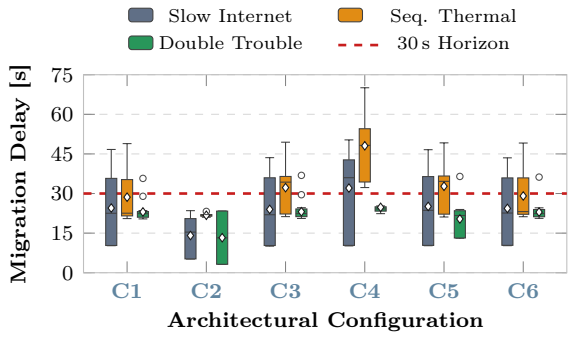


Figure 3: Migration delay comparison: extracting the entire monolithic container (C_4) exceeds the 30 s safety horizon (dashed red line). Isolating the stateful sidecar (C_1) remains below the safety horizon in the targeted comparison.

6.4. Phase D: Logarithmic Utility Routing

Under severe dual stress, the linear scoring function (C5) allowed hop distance to mask energy depletion, routing to a satellite with only 5.5% battery and crashing. C1's logarithmic barrier formulation addressed this: as battery approaches depletion, the penalty grows to infinity, safely aborting migration and yielding a 100% survival rate in this scenario.

6.5. Phase E: Sequenced Evacuation

When severe thermal distress forced both a worker and the Master to move simultaneously, uncoordinated concurrent migration (C6) succeeded only 28.57% of the time, compared to 100% for C1 in the tested dual-evacuation trials. Under C6, the Master suspended routing while the worker was requesting an escape route, causing an orchestration deadlock. Table 1 reports the overall survival matrix.

6.6. End-to-End Endurance and Operational Stability

While the preceding ablation phases stress-test individual architectural components over short bursts, a continuous run evaluates whether the full C1 configuration maintains stable operation over the full 23-minute uninterrupted evaluation window. The platform underwent an uninterrupted 23-minute continuous run using the 2021 wildfire dataset, comprising 50 concurrent fire time-series across 63 logical observation passes. The inference engine ran ONNX inference under realistic CPU load. The orchestration layer executed without encountering a runtime failure. The platform autonomously initiated proactive migrations, preserving temporal history buffers across all 50 tracking missions, maintaining complete numerical precision without pipeline stalls. These results complement the factorial matrix in Table 1 by providing evidence that the per-trial reliability observed in the 540-trial campaign extends to continuous operation across multiple simulated orbital cycles.

7. Conclusions and Future Work

Space Cloud successfully migrates running containers between emulated LEO satellites to

	Scenario 1 Link Degradation			Scenario 2 Sequential Thermal			Scenario 3 Double Trouble		
Cfg	Nom	Bord	Fail	Nom	Bord	Fail	Nom	Bord	Fail
C1	1.00	0.50	0.00	1.00	1.00	1.00	1.00	0.60	0.00
C2	0.80	0.00	0.00	0.50	0.30	0.00	1.00	0.00	0.00
C3	1.00	0.70	0.00	1.00	1.00	1.00	1.00	0.70	0.00
C4	1.00	0.00	0.00	1.00	0.00	1.00	1.00	0.00	0.00
C5	1.00	0.40	0.00	1.00	1.00	1.00	1.00	0.80	0.00
C6	1.00	0.60	0.10	0.40	0.60	1.00	1.00	0.60	0.00

Table 1: Physical survival rates across the $6\times3\times3$ factorial matrix (540 trials). Shading indicates resilience level. C_1 achieved full survival across all nominal and sequential-thermal conditions.

maintain application uptime. Splitting the stateful sidecar from the stateless model runner reduces checkpoint sizes by up to 85%. Forecasting satellite conditions 30 seconds ahead improves workload survival from 5.00% to 51.67% under severe stress. Extracting process state via CRIU preserves historical observation frames with zero data loss. Evaluating targets logarithmically avoids routing to depleted nodes, achieving a 100% safe-abort outcome under severe dual stress, while moving shared pods sequentially prevents orchestration deadlocks.

While empirical evaluation supports that Space Cloud preserves stateful continuity under severe orbital stress, translating this architecture to flight-qualified LEO missions exposes several operational boundaries that guide future work:

Constellation Scalability. Centralized pathfinding performs in sub-millisecond time on three satellites, but a 3,000-node fleet with 12,000 dynamic links creates severe bottlenecks. Future work aims to scale to massive constellations by replacing the Floating Master with a decentralized consensus cluster like Raft.

Thermal Model Fidelity. The lumped-mass model omits Earth albedo and die hotspots. Real spacecraft require fine-grained throttling alongside migration, and future physical simulations must incorporate Earth albedo and component-level thermal tracking.

Radiation Vulnerability. Terrestrial DRAM lacks SEU protection. CRIU dumps RAM directly; without ECC scrubbing, bit flips propagate to healthy nodes. Future systems must combine state saving with ECC memory checks.

Transport Protocol Fragility. Standard TCP misinterprets orbital noise and packet loss as congestion, collapsing throughput. The networking layer must implement Delay-Tolerant Networking with custody transfer to handle Doppler shifts and 7.5 km/s relative velocities.

Workload Sizing Limits. Validations used a 24 MB ResNet-18 sidecar. Workloads producing 200 MB checkpoints breach the 30-second thermal window. Supporting massive foundation models requires incremental checkpointing or live data compression to reliably finish within strict thermal limits.

8. Acknowledgements

I express my sincere gratitude to my advisor, Professor Emanuele Della Valle, for his guidance, support, and encouragement.

References

[1] Emil P. Andersen, Ola Vanni Flaata, and Frank T. Johnsen. Edge computing with Kubernetes: KubeEdge in 5G, SATCOM, and tactical networks. In *Proceedings of the 12th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 2025.

[2] Adam Duggan, Bruno Andrade, and Haithem Afli. Advancing earth observation: A survey on AI-powered image processing in satellites. 2025.

[3] Sebastian Gerard, Yu Zhao, and Josephine Sullivan. WildfireSpreadTS: A dataset of multi-modal time series for wildfire spread prediction. 36:74515–74529, 2023.

[4] Minghao Hui, Shaohua Zhai, Dongming Wang, Tong Hui, Wei Wang, Peng Du, and Fengkui Gong. A review of LEO-satellite communication payloads for integrated communication, navigation, and remote sensing: Opportunities, challenges, future directions. *IEEE Internet of Things Journal*, 12(12):18954–18992, 2025.

[5] Xinyu Wang, Xiaolin Ju, Renchao Xie, Qinqin Tang, Tao Chen, and Tao Huang. Service continuity guarantee for coordinated optimization of offloading and migration in LEO satellite computing power networks. In *Proceedings of the 10th International Conference on Computer and Communications (ICCC)*, pages 1857–1862, 2024.